NAMA: IKLINA NAJZIL MUHSININA

NIM: 242410101001

🔬 AKTIVITAS MANDIRI 1 — Instalasi & Setup Proyek Laravel

1. Instal XAMPP terbaru, pastikan PHP 8.2+ tersedia (cek: php --version di terminal)

```
C:\laragon\www
λ php -v
PHP 8.3.30 (cli) (built: Jan 13 2026 22:50:40) (ZTS Visual C++ 2019 x64)
Copyright (c) The PHP Group
Zend Engine v4.3.30, Copyright (c) Zend Technologies
```

⇒ Dikarenakan saya sudah menginstall laragon, jadi saya tidak menginstall Xampp. Untuk gambar diatas menunjukkan proses verifikasi instalasi PHP melalui terminal pada direktori kerja Laragon (C:\laragon\www). Dengan menjalankan perintah php -v, sistem memberikan informasi bahwa perangkat tersebut telah terpasang PHP versi 8.3.30 yang dirilis pada Januari 2026. Keterangan tersebut juga mengonfirmasi penggunaan Zend Engine v4.3.30 dan arsitektur x64, yang menandakan bahwa lingkungan pengembangan web sudah siap digunakan untuk menjalankan skrip PHP maupun framework modern dengan stabil.

2. Instal Composer dari getcomposer.org, verifikasi dengan: composer --version

```
C:\laragon\www
λ composer -v

   ______
  / ____/___  ____ ___  ____  ____  ________  _____
 / /   / __ \/ __ `__ \/ __ \/ __ \/ ___/ _ \/ ___/
/ /___/ /_/ / / / / / / /_/ / /_/ (__  )  __/ /
\____/\____/_/ /_/ /_/ .___/\____/____/\___/_/
                    /_/
Composer version 2.9.4 2026-01-22 14:08:50
```

⇒ Untuk Composernya telah berhasil diinstall. Melalui perintah `composer -v` di dalam terminal, muncul logo teks khas Composer beserta keterangan versi 2.9.4.

3. Buat proyek Laravel baru: composer create-project laravel/laravel siak-app

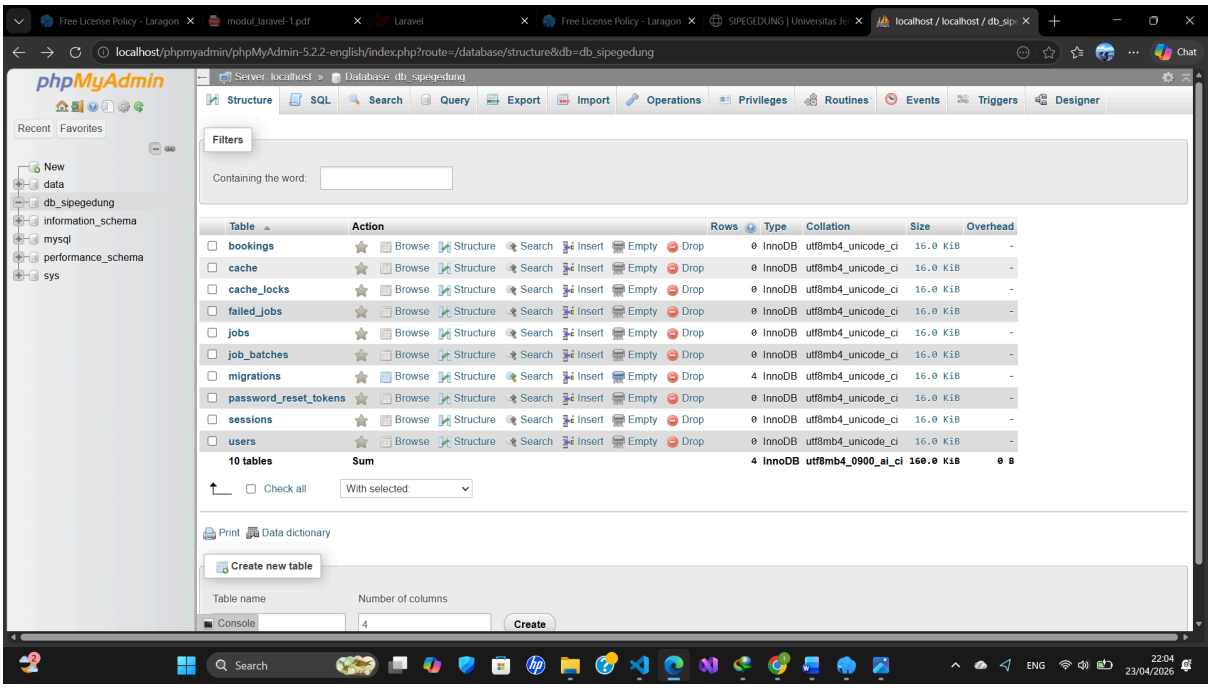
```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS    DEVDB

PS C:\laragon\www> composer create-project laravel/laravel siak-app
Creating a "laravel/laravel" project at "./siak-app"
Installing laravel/laravel (v13.3.0)
  - Installing laravel/laravel (v13.3.0): Extracting archive
Created project in C:\laragon\www\siak-app
> @php -r "file_exists('.env') || copy('.env.example', '.env');"
Loading composer repositories with package information
Updating dependencies
Lock file operations: 107 installs, 0 updates, 0 removals
  - Locking brick/math (0.14.8)
  - Locking carbonphp/carbon-doctrine-types (3.2.0)
  - Locking dflydev/dot-access-data (v3.0.3)
  - Locking doctrine/inflector (2.1.0)
  - Locking doctrine/lexer (3.0.1)
  - Locking dragonmantank/cron-expression (v3.6.0)
  - Locking egulias/email-validator (4.0.4)
  - Locking fakerphp/faker (v1.24.1)
  - Locking filp/whoops (2.18.4)
  - Locking fruitcake/php-cors (v1.4.0)
  - Locking graham-campbell/result-type (v1.1.4)
  - Locking guzzlehttp/guzzle (7.10.0)
  - Locking guzzlehttp/promises (2.3.0)
  - Locking guzzlehttp/psr7 (2.9.0)
  - Locking guzzlehttp/uri-template (v1.0.5)
  - Locking hamcrest/hamcrest-php (v2.1.1)
  - Locking laravel/framework (v13.6.0)
  - Locking laravel/pail (v1.2.6)
  - Locking laravel/pint (v1.29.1)
  - Locking laravel/prompts (v0.3.17)
  - Locking laravel/serializable-closure (v2.0.12)
  - Locking laravel/tinker (v3.0.2)
  - Locking league/commonmark (2.8.2)
  - Locking league/config (v1.2.0)
  - Locking league/flysystem (3.33.0)
  - Locking league/flysystem-local (3.31.0)
  - Locking league/mime-type-detection (1.16.0)
  - Locking league/uri (7.8.1)
  - Locking league/uri-interfaces (7.8.1)
  - Locking mockery/mockery (1.6.12)
  - Locking monolog/monolog (3.10.0)
  - Locking myclabs/deep-copy (1.13.4)
  - Locking nesbot/carbon (3.11.4)
  - Locking nette/schema (v1.3.5)
  - Locking nette/utils (v4.1.3)
  - Locking nikic/php-parser (v5.7.0)
  - Locking nunomaduro/collision (v8.9.4)
  - Locking nunomaduro/termwind (v2.4.0)
  - Locking phar-io/manifest (2.0.4)
  - Locking phar-io/version (3.2.1)
  - Locking phpoption/phpoption (1.9.5)
```

```
  - Installing phpunit/php-text-template (5.0.0): Extracting archive
  - Installing phpunit/php-invoker (6.0.0): Extracting archive
  - Installing phpunit/php-file-iterator (6.0.1): Extracting archive
  - Installing theseer/tokenizer (2.0.1): Extracting archive
  - Installing sebastian/lines-of-code (4.0.0): Extracting archive
  - Installing sebastian/complexity (5.0.0): Extracting archive
  - Installing phpunit/php-code-coverage (12.5.6): Extracting archive
  - Installing phar-io/version (3.2.1): Extracting archive
  - Installing phar-io/manifest (2.0.4): Extracting archive
  - Installing myclabs/deep-copy (1.13.4): Extracting archive
  - Installing phpunit/phpunit (12.5.23): Extracting archive
60 package suggestions were added by new dependencies, use `composer suggest` to see details.
Generating optimized autoload files
> Illuminate\Foundation\ComposerScripts::postAutoloadDump
> @php artisan package:discover --ansi

  INFO  Discovering packages.

  laravel/pail ..................................................... DONE
  laravel/tinker ................................................... DONE
  nesbot/carbon .................................................... DONE
  nunomaduro/collision ............................................. DONE
  nunomaduro/termwind .............................................. DONE

78 packages you are using are looking for funding.
Use the `composer fund` command to find out more!
> @php artisan vendor:publish --tag=laravel-assets --ansi --force

  INFO  No publishable resources for tag [laravel-assets].

No security vulnerability advisories found.
> @php artisan key:generate --ansi

  INFO  Application key set successfully.

> @php -r "file_exists('database/database.sqlite') || touch('database/database.sqlite');"
> @php artisan migrate --graceful --ansi

  INFO  Preparing database.

  Creating migration table ..................................... 45.74ms DONE

  INFO  Running migrations.

  0001_01_01_000000_create_users_table ......................... 70.70ms DONE
  0001_01_01_000001_create_cache_table ......................... 63.68ms DONE
  0001_01_01_000002_create_jobs_table .......................... 75.14ms DONE

PS C:\laragon\www> cd siak-app
PS C:\laragon\www\siak-app>
```

⇒ Gambar diatas merupakan proses pembuatan proyek baru menggunakan framework Laravel dengan nama "siak-app" melalui perintah composer create-project. Proses ini mencakup pengunduhan berbagai paket dependensi, pengaturan kunci aplikasi (app

key), hingga menjalankan migrasi database otomatis untuk membuat tabel dasar seperti tabel pengguna dan pekerjaan. Di akhir tampilan, terlihat bahwa proyek telah berhasil dibuat dan terminal sudah masuk ke dalam direktori proyek tersebut, menandakan struktur dasar website siap untuk mulai dikembangkan.

4. Buat database baru 'siak_db' di phpMyAdmin



⇒ Gambar diatas menunjukkan proses pembuatan database baru bernama db_sipegedung melalui antarmuka phpMyAdmin. Langkah tersebut kemudian akan dihubungkan dengan proyek aplikasi web yang sedang dibangun.

5. Konfigurasi file .env: isi DB_DATABASE, DB_USERNAME, DB_PASSWORD sesuai lokal

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=db_sipegedung
DB_USERNAME=root
DB_PASSWORD=123456
```

⇒Gambar diatas merupakan potongan kode dari file konfigurasi lingkungan yang ada di file .env yang berfungsi untuk menghubungkan aplikasi dengan database. Di dalamnya diatur parameter penting seperti jenis koneksi menggunakan MySQL, alamat host lokal (127.0.0.1), serta nama database yang dituju yaitu db_sipegedung. Pengaturan tersebut memastikan aplikasi memiliki izin akses yang tepat melalui username dan password yang telah ditentukan agar sistem dapat menyimpan maupun mengambil data dari server database secara sinkron.

6. Jalankan: php artisan key:generate

```
PS C:\laragon\www\siak-app> php artisan key:generate

  INFO  Application key set successfully.

PS C:\laragon\www\siak-app>
```

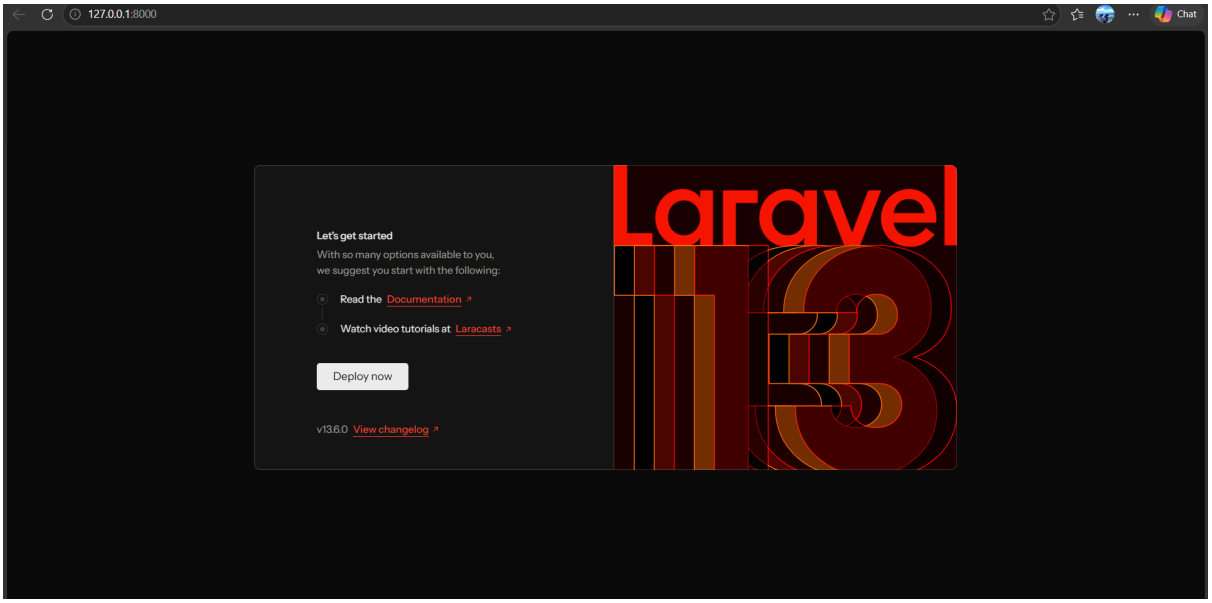
⇒ Perintah diatas berfungsi untuk membuat kunci enkripsi unik yang akan disimpan di dalam file .env. Pesan "Application key set successfully" menandakan bahwa kunci keamanan telah berhasil dibuat, yang sangat penting untuk melindungi data sensitif seperti session pengguna dan password yang terenkripsi agar aplikasi berjalan dengan aman. dan nanti dibagian .env akan berubah seperti di bawah ini.

```
APP_NAME=Laravel
APP_ENV=local
APP_KEY=base64:NBw8szzyKKOss7ENhejv4iY2s/iXUGuKCpkuVL2t9Fs=
APP_DEBUG=true
APP_URL=http://localhost
```

7. Jalankan server: php artisan serve — buka http://127.0.0.1:8000 di browser

```
PS C:\laragon\www\siak-app> php artisan serve

  INFO  Server running on [http://127.0.0.1:8000].

  Press Ctrl+C to stop the server

  2026-04-28 14:00:20 / ..............................................
```

Kemudian menjalankan server dengan perintah php artisan serve kemudian akan muncul http://127.0.0.1:8000, dan jika di pencet akan menampilkan Web yang kita buat, dan jika proyeknya belum membuat file view, maka tampilannya seperti ini



8. Eksplorasi struktur folder, buka setiap folder dan pahami fungsinya
9. Jalankan php artisan route:list dan catat route apa saja yang sudah terdaftar

```
PS C:\laragon\www\siak-app> php artisan route:list

  GET|HEAD  / ..................................................... routes/web.php:5
  GET|HEAD  storage/{path} ..... storage.local › vendor/laravel/framework/src/Illuminate/Filesystem/FilesystemServiceProvider.php:111
  PUT       storage/{path} ..... storage.local.upload › vendor/laravel/framework/src/Illuminate/Filesystem/FilesystemServiceProvider.php:119
  GET|HEAD  up ..... vendor/laravel/framework/src/Illuminate/Foundation/Configuration/ApplicationBuilder.php:223

                                                                                                  Showing [4] routes

PS C:\laragon\www\siak-app>
```

⇒ Setelah itu mengeksekusi perintah php artisan route:list yang digunakan untuk menampilkan daftar alamat URL atau route yang tersedia di dalam aplikasi Laravel. Hasilnya memperlihatkan empat buah *route* aktif, termasuk alamat utama (/) yang diatur pada file routes/web.php, serta beberapa *route* otomatis bawaan framework untuk kebutuhan akses penyimpanan file (*storage*) dan pengecekan kesehatan sistem (*up*). Penjelasan ini berfungsi untuk memetakan jalur akses mana saja yang bisa dibuka oleh pengguna melalui browser atau digunakan oleh aplikasi untuk berkomunikasi dengan server.

🔬 AKTIVITAS MANDIRI 2 — Membuat Routing & Controller Dashboard

1. Buat Controller DashboardController: php artisan make:controller DashboardController

```
PS C:\laragon\www\siak-app>  php artisan make:controller DashboardController

  INFO  Controller [C:\laragon\www\siak-app\app\Http\Controllers\DashboardController.php] created successfully.

PS C:\laragon\www\siak-app>
```

⇒ Gambar tersebut menunjukkan proses pembuatan Controller baru bernama `DashboardController` menggunakan perintah `php artisan make:controller`. Berhasilnya eksekusi perintah ini ditandai dengan pesan sukses yang menginformasikan bahwa file tersebut telah dibuat secara otomatis di dalam direktori `app/Http/Controllers/`. Controller ini nantinya akan berfungsi sebagai otak yang mengatur logika bisnis aplikasi, seperti menghubungkan data dari database ke tampilan antarmuka (view) pada halaman dashboard.

2. Tambahkan method index() yang mengembalikan view('dashboard') di DashboardController

```
.env   DashboardController.php ×   web.php   tentang.blade.php
siak-app > app > Http > Controllers > DashboardController.php
1   <?php
2
3   namespace App\Http\Controllers;
4
5   use Illuminate\Http\Request;
6
7   class DashboardController extends Controller
8   {
9       public function index(){
10          return view('dashboard');
11      }
12  }
```

⇒ Gambar tersebut menunjukkan isi dari file DashboardController.php. Di dalamnya terdapat sebuah fungsi (method) bernama index() yang hanya memiliki satu tugas utama, yaitu memerintahkan aplikasi untuk menampilkan halaman dashboard.

Melalui baris kode return view('dashboard');, Controller akan mencari file bernama dashboard.blade.php di dalam folder views untuk dikirimkan dan ditampilkan ke browser

pengguna. Dalam arsitektur MVC (Model-View-Controller), file ini bertindak sebagai penghubung sederhana yang memastikan permintaan pengguna untuk melihat halaman dashboard langsung diarahkan ke tampilan yang tepat.

3. Daftarkan route di routes/web.php: Route::get('/dashboard', [DashboardController::class, 'index'])->name('dashboard')

```php
<?php

use Illuminate\Support\Facades\Route;
use App\Http\Controllers\DashboardController;

Route::get('/dashboard', [DashboardController::class, 'index'])->name('dashboard');
```

⇒ Kemudian dilanjutkan pendaftaran route di dalam file routes/web.php untuk menghubungkan URL dengan logika aplikasi. Melalui baris kode Route::get('/dashboard', ...), sistem diatur agar ketika alamat /dashboard diakses di browser, aplikasi akan menjalankan fungsi index yang ada di dalam DashboardController. Penambahan baris use App\Http\Controllers\DashboardController; di bagian atas sangat penting agar sistem mengenali lokasi file controller tersebut. Selain itu, penggunaan ->name('dashboard') berfungsi untuk memberi nama unik pada route, sehingga memudahkan pemanggilan alamat URL tersebut di bagian lain dari kode program tanpa harus menuliskan alamat lengkapnya secara manual

4. Buat file view sementara resources/views/dashboard.blade.php berisi '<h1>Dashboard SIAK</h1>'

```
.env   DashboardController.php   web.php   dashboard.blade.php ×
siak-app > resources > views > dashboard.blade.php > ...
1   <h1>Dashboard SIAK</h1>
2
```

⇒ Kemudian dilanjutkan dengan membuat file tampilan dasar bernama dashboard.blade.php di dalam folder resources/views. File ini hanya berisi kode HTML sederhana (tag <h1>) sebagai tanda bahwa halaman dashboard berhasil dimuat.

5. Akses http://127.0.0.1:8000/dashboard di browser — pastikan tampil

⇒ Lalu melakukan verifikasi dengan mengakses alamat http://127.0.0.1:8000/dashboard di browser. Jika muncul tulisan "Dashboard SIAK", berarti koneksi antara route, controller, dan view sudah berjalan dengan benar.

6. Buat route /tentang dengan Route::view('/tentang', 'tentang') dan buat view-nya

```php
<?php

use Illuminate\Support\Facades\Route;
use App\Http\Controllers\DashboardController;

Route::get('/dashboard', [DashboardController::class, 'index'])->name('dashboard');
Route::view('/tentang', 'tentang');
```

```html
<!DOCTYPE html>
<html>
<head>
    <title>Halaman Tentang</title>
</head>
<body>
    <h1>Ini adalah Halaman Tentang Kami</h1>
    <p>Selamat datang di aplikasi siak-app.</p>
</body>
</html>
```

⇒ Dilanjutkan dengan menambahkan route baru untuk halaman "Tentang" menggunakan metode Route::view yang lebih praktis, serta membuat file tentang.blade.php. Ini digunakan untuk menampilkan konten statis yang tidak memerlukan logika rumit dari controller.

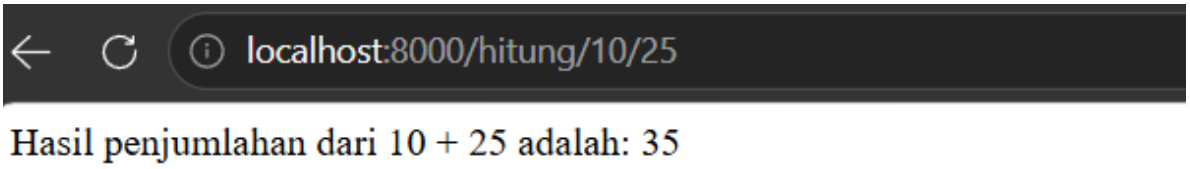
7. Jalankan php artisan route:list — amati semua route yang terdaftar

```
PS C:\laragon\www\siak-app> php artisan route:list

  GET|HEAD  dashboard ..... dashboard › DashboardController@index
  GET|HEAD  storage/{path} ..... storage.local › vendor/laravel/framework/src/Illuminate/Filesystem/FilesystemServiceProvider.php:111
  PUT       storage/{path} ..... storage.local.upload › vendor/laravel/framework/src/Illuminate/Filesystem/FilesystemServiceProvider.php:119
  GET|HEAD  tentang .....
  GET|HEAD  up ..... vendor/laravel/framework/src/Illuminate/Foundation/Configuration/ApplicationBuilder.php:223

                                                                                                  Showing [5] routes

PS C:\laragon\www\siak-app>
```

⇒ Kemudian menjalankan perintah php artisan route:list di terminal untuk memantau seluruh daftar alamat URL yang telah terdaftar di sistem. Langkah ini memastikan bahwa alamat /dashboard dan /tentang sudah dikenali oleh aplikasi dan siap digunakan.

8.Coba buat Route::get('/hitung/{a}/{b}', fn($a, $b) => $a + $b) dan akses /hitung/10/25

```php
// Route dinamis untuk menghitung dua angka
Route::get('/hitung/{a}/{b}', function($a, $b) {
    $hasil = $a + $b;
    return "Hasil penjumlahan dari $a + $b adalah: " . $hasil;
});
```



Hasil penjumlahan dari 10 + 25 adalah: 35

⇒ Untuk poin 8 dalam menjelaskan tentang penerapan Dynamic Routing atau rute dinamis pada Laravel dengan menggunakan parameter URL. Dalam kode

Route::get('/hitung/{a}/{b}', fn($a, $b) => $a + $b), tanda kurung kurawal {a} dan {b} berfungsi sebagai variabel penampung (placeholder) yang akan menangkap nilai apa pun yang diketikkan pengguna di alamat browser setelah bagian /hitung/. Saat pengguna mengakses URL seperti /hitung/10/25, Laravel secara otomatis mengambil angka 10 untuk mengisi variabel $a dan angka 25 untuk variabel $b.

Proses ini kemudian dilanjutkan dengan eksekusi fungsi singkat atau arrow function (fn) yang menjumlahkan kedua nilai tersebut, sehingga browser akan langsung menampilkan angka 35 sebagai hasilnya. Secara teknis, poin ini bertujuan untuk menunjukkan betapa fleksibelnya Laravel dalam mengelola data dari URL secara real-time, yang nantinya sangat berguna untuk fitur-fitur seperti pencarian data spesifik berdasarkan ID atau kategori dalam sistem informasi yang sedang kamu bangun.